

HCM 实施助手

项目讲解文档

—— 一份给未来的自己看的代码地图

版本: v2.5

作者: 宋庚 (Geng Song)

写于: 2026 年 5 月

GitHub: github.com/songgeng758-web/hcm-assistant

前言

这份文档不是技术规范、不是 API 手册、也不是教程。它是我在做完 HCM 实施助手这个项目后，把自己脑子里的东西梳理出来给「将来某天回看代码的自己」看的一份地图。

我写过几次项目，但都有一个通病：写完没多久就忘了某段代码为什么这么写。等到几个月后回来想改点东西，就要花大量时间重新理解自己写过的代码。这次我决心把「为什么」写下来，而不只是「是什么」。

所以这份文档的特点是：每讲一段代码，我会同时讲清楚它解决什么业务问题、为什么选这种实现方式、有什么替代方案、踩过什么坑。

代码会过时，但解决问题的思路不会。这份文档记录的是后者。

如果你是路过 GitHub 偶然点进来的读者，希望它对你有一点点参考价值。如果你是 HCM 行业的同行，欢迎通过 GitHub 联系我交流。

第一章 项目总览

1.1 这是个什么项目

HCM 实施助手是一个面向 HCM（人力资本管理）系统实施顾问的 Web 端工具，帮助实施顾问处理客户数据迁移和校验过程中的各种重复性工作。

整个项目围绕三个真实业务场景设计：

- **员工档案校验**——把客户给的脏 Excel 校验干净，识别字段错误、格式异常、一致性问题
- **组织架构解析**——把扁平的员工表自动还原成多级组织架构树
- **薪酬计算**——单人工资试算 + 批量算薪，覆盖五险一金、个税完整链路

1.2 项目数据

做完后客观盘点一下，这个项目的体量：

指标	数据
后端 Python 文件	约 15 个
前端 Vue 文件	7 个（5 个页面 + 主框架 + 路由）
API 接口数量	15 个
单元测试数量	40+ 个（全部通过）
真实数据验证规模	6017 行人员主集
代码总行数	约 2500+ 行
开发周期	约 2 周（入职空窗期）

1.3 技术栈选择

后端技术栈

技术	版本	选它的理由
----	----	-------

Python	3.11+	主流、生态成熟、pandas 等数据处理库无可替代
FastAPI	0.115	自动生成 Swagger 文档，类型校验自带
Pydantic	2.x	数据校验，让接口请求和响应都有「形状」约束
pandas	2.2	Excel 读写、数据清洗的事实标准
openpyxl	3.1	生成带样式的 Excel 报告
pytest	8.x	单元测试

前端技术栈

技术	版本	选它的理由
Vue 3	3.x	国内最普及的前端框架之一，Composition API 风格清晰
Vite	5.x	启动快、热更新极快，比 webpack 更现代
Element Plus	2.x	企业级中后台 UI 的事实标准
Vue Router	4.x	单页应用路由
Axios	1.x	HTTP 请求库

为什么选 FastAPI 不选 Django

做实施工具，Django 太重——它带着模板、ORM、Admin 后台等一整套全栈方案。我做的是纯 API 服务，前后端完全分离，Django 的优势用不上。

FastAPI 的几个关键优势恰好都是我需要的：

- **自动 Swagger 文档**——写代码即写文档，前端联调时直接用 /docs 调试
- **类型校验自带**——Pydantic 直接做请求/响应校验，少了一层手写校验逻辑
- **异步原生支持**——文件上传场景受益明显

- **学习曲线平缓**——比 Django 更简单直接，适合一个人项目

第二章 架构设计

2.1 整体架构：前后端分离

整个项目是标准的前后端分离架构。后端只管数据和计算，前端只管展示，两者靠 HTTP API 通信。



这种架构的好处：

- 后端可以独立部署，未来想给微信小程序或其他客户端用也方便
- 前端可以独立迭代 UI，不影响业务逻辑
- 前后端可以分别用各自的最佳工具链（Python 处理数据 + Vue 做界面）

2.2 后端目录结构

```
backend/  
├─ app/  
│   └─ api/                # API 路由层（薄薄一层）  
│       └─ org.py          # 组织人事接口  
│       └─ payroll.py      # 薪酬计算接口
```

```

├── common.py
├── core/
│   ├── config.py          # 全局配置
│   └── modules/           # 业务逻辑层（核心算法）
│       ├── org/
│           ├── employee_validator.py # 员工档案校验
│           ├── org_tree_parser.py   # 组织架构解析
│       └── payroll/
│           ├── calculator.py        # 薪酬计算引擎
│           └── batch_calculator.py  # 批量算薪
├── schemas/               # Pydantic 数据模型
├── utils/                 # 工具函数
│   ├── id_card.py         # 身份证算法
│   └── validators.py       # 通用校验器
├── __init__.py
├── tests/                 # 单元测试
├── data/                 # 数据目录（运行时生成）
├── main.py                # FastAPI 入口
└── requirements.txt

```

分层设计的核心思想

我刻意把代码分成了三层：

- **API 层 (api/)** ——只负责处理 HTTP 请求、解析参数、调用业务逻辑、返回响应。极薄。
- **业务层 (modules/)** ——所有核心算法和业务逻辑都在这里。不依赖 Web 框架，纯函数式。
- **工具层 (utils/)** ——可复用的工具，比如身份证算法、各种校验器

这样做的好处：业务逻辑跟 HTTP 解耦。哪天我想把这个工具改成命令行版本、定时任务、或者集成到其他系统，只要换掉 API 层就行，业务层完全不用动。

这也是为什么我的单元测试可以直接测业务函数（calculate_payroll、validate_employees 等），不需要启动 FastAPI 服务。

第三章 员工档案校验模块详解

3.1 业务背景：为什么客户的数据永远是脏的

HCM 实施的第一步几乎都是数据迁移——把客户老系统里的员工数据导入新的 HCM。客户提供的「员工主集」Excel 看起来简单，实际上每次都有意外。常见的脏数据场景：

- **字段名不规范**——客户用「员工编号」、老系统导出叫「工号」、行政人员手填的叫「工号/编号」
- **Excel 自动转换破坏数据**——18 位身份证号被自动转成科学计数法、电话号码 0 开头被吃掉
- **表头藏脏字符**——空格、换行符、零宽字符（看不见但是有）
- **字段一致性错乱**——身份证显示是男的，但性别列填了女（行政误录）
- **工号重复**——分公司各自编号导致冲突、人员调动后没去重

员工档案校验模块就是为了系统化解决这些问题而生。

3.2 核心算法：身份证解析

身份证号的结构

中国大陆 18 位居民身份证号码遵循 GB 11643-1999 标准，每一位都有含义：



校验码算法

最后一位校验码不是随便填的，它是根据前 17 位计算出来的。算法：

- 把前 17 位每一位的数字，乘以对应位置的加权因子
- 把所有乘积加起来

- 除以 11 取余数，余数对应一个字符表，就是校验码

加权因子和校验码表是固定的：

```
WEIGHTS = [7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2]
CHECK_CODES = ['1', '0', 'X', '9', '8', '7', '6', '5', '4', '3', '2']

def calc_check_code(id_17: str) -> str:
    total = sum(int(ch) * w for ch, w in zip(id_17, WEIGHTS))
    return CHECK_CODES[total % 11]
```

有了这个算法，给一个完整的 18 位号码就能验证它的合法性——拿前 17 位算出校验码，跟实际的第 18 位对比。

注意：第 18 位可能是 X (罗马数字 10)，代表余数为 2 的情况。所以处理时要把字符串先 upper() 统一大小写。

从身份证号能解析出什么

一个合法的身份证号，我们可以解析出：

- **出生日期**——直接从第 7-14 位读，注意要校验日期合法性（比如 2 月 30 日就不合法）
- **性别**——第 17 位（顺序码末位），奇数为男，偶数为女
- **年龄**——用今天的日期减去出生日期。要注意「生日是否已过」的判断

年龄计算的小坑

年龄的计算不是简单的「今年减去出生年」。比如：2026 年 5 月 22 日，一个 1990 年 3 月 7 日出生的人和一个 1990 年 8 月 8 日出生的人，虚岁都是 36，但实岁前者是 36、后者还是 35（因为还没过 8 月 8 日生日）。

```
# 正确的实岁计算
age = today.year - birth.year - (
    (today.month, today.day) < (birth.month, birth.day)
)

# 当 (today.month, today.day) 还没到生日时，
# Python 的元组比较返回 True，
```

```
# True 在算术运算里被当作 1，所以年龄少 1。
```

这一行代码是面试时常考的小技巧。看似简单，但很多新手会写成 `age = today.year - birth.year`，导致前半年的所有数据年龄都偏大 1 岁。

3.3 关键设计：字段别名机制

实施顾问的痛点是客户字段名永远不一致。我设计了一个「字段别名表」机制：

```
FIELD_ALIASES = {
    'employee_id': ['工号', '员工编号', '员工号', 'employee_id', '工号/编号'],
    'name': ['姓名', 'name', '员工姓名'],
    'id_card': ['身份证号', '身份证', '证件号码', '证件号', 'id_card'],
    'gender': ['性别', 'gender'],
    'phone': ['手机号', '电话', '联系电话', 'phone', 'mobile'],
    # ...
}
```

解析 Excel 时，程序对每个标准字段都尝试匹配别名表里的多种写法。这样客户表里叫「工号」还是「员工编号」，程序都能识别。

表头清洗：处理「看不见的脏东西」

客户从老系统导出的 Excel，表头里经常藏着各种「看不见的字符」：

```
def clean_header(value: str) -> str:
    '''清洗 Excel 表头'''
    if value is None:
        return ''
    s = str(value)
    invisible_chars = [
        '\u00a0', # 不间断空格
        '\u200b', # 零宽空格
        '\u200c', # 零宽不连字符
        '\u200d', # 零宽连字符
        '\uffff', # BOM
        '\t', '\n', '\r', # 制表符、换行
    ]
    for ch in invisible_chars:
        s = s.replace(ch, '')
    return s.strip()
```

这段代码看起来很普通，但它是从一个真实失败案例里「逼」出来的——第一次跑校验时，明明 Excel 里有「工号」这一列，程序就是识别不出来。Debug 后才发现客户的表头叫「工

号」，结尾多了一个不间断空格。这种问题肉眼看不出来，但 Python 严格比对就会出错。

实施顾问的核心能力之一就是「对脏数据保持警觉」。表头空格只是冰山一角。

3.4 校验流程详解

整个校验函数的流程：

1. 读取 Excel（关键：用 dtype=str 强制按字符串读，避免 pandas 自动类型转换破坏身份证号、工号等字段）

```
df = pd.read_excel(file_path, dtype=str)

# dtype=str 至关重要：
# - 防止 18 位身份证被转成 1.10E+17 科学计数法
# - 防止 0 开头的工号丢失前导 0
# - 防止数字日期被转成 Timestamp 对象
```

2. 基础清洗：去空行、去空列、字符串字段去首尾空格
3. 字段映射：用别名表识别每个标准字段对应的实际列名
4. 逐行校验：必填字段、格式校验、唯一性、一致性
5. 生成异常列表（带行号、字段名、级别、消息、建议）
6. 输出校验报告 Excel

错误分级：error vs warning

一个重要的设计：所有异常都分两级

级别	含义	示例
error	阻断性问题，必须修复才能导入	工号为空、身份证错、性别与身份证不符
warning	可接受但建议修复的问题	手机号格式不对、邮箱格式不对

这个分级不是技术决定，是业务决定——身份证错了，HCM 系统肯定导不进；手机号错了，

不影响入库，只影响以后发通知。实施顾问要根据级别决定优先处理哪些异常。

3.5 行号定位的重要性

每条异常都带 Excel 行号，方便实施顾问对照原文件修复：

```
for idx, row in df.iterrows():  
    # Excel 行号 = pandas 行号 + 2  
    # +1 因为 pandas 是 0-based  
    # +1 因为 Excel 第 1 行是表头  
    excel_row = idx + 2
```

一个「+2」的细节，决定了实施顾问拿到报告后是直接打开 Excel 跳到对应行修复，还是要手工搜索定位。

第四章 组织架构解析模块详解

4.1 业务背景：扁平表到树的转换

HCM 系统需要的是树形组织结构（集团 → 中心 → 部门 → 组），但客户给的人员表是扁平的——每一行员工自带「一级组织」到「十级组织」的完整路径。

比如客户表里可能是这样：

员工	一级组织	二级组织	三级组织	四级组织
张三	集团总部	财务中心	会计部	应收会计组
李四	集团总部	财务中心	会计部	应付会计组
王五	集团总部	财务中心	出纳部	
赵六	分公司 A	综合部		

我们要把它还原成树：

```
集团总部
├── 财务中心
│   ├── 会计部
│   │   ├── 应收会计组（1 人：张三）
│   │   └── 应付会计组（1 人：李四）
│   └── 出纳部（1 人：王五）
└── 分公司 A
    └── 综合部（1 人：赵六）
```

4.2 核心算法：扁平转树

算法的核心思路是「用 dict 嵌套作为树容器，逐行扫描，把每条路径插入树」：

```
# 树的内部表示：嵌套 dict
tree = {
    '_count': 0,
    '_level': 0,
    'children': {
        '集团总部': {
            '_count': 0,
            '_level': 1,
            'children': {
                '财务中心': {
                    '_count': 0,
                    '_level': 2,
                    'children': { }
                }
            }
        }
    }
}
```

```

    }
  }
}

```

插入算法：

```

def _insert_to_tree(tree: dict, path: List[str]) -> dict:
    '''把一条路径插入树，返回叶节点'''
    current = tree
    for level, name in enumerate(path, start=1):
        # 如果当前节点的 children 里没有这个部门名，就创建一个
        if name not in current['children']:
            current['children'][name] = {
                '_count': 0,
                '_level': level,
                'children': {},
            }
        # 下沉到子节点
        current = current['children'][name]
    # 循环结束时，current 就是叶节点
    return current

```

时间复杂度 $O(\text{行数} \times \text{层级数})$ 。对于 6017 行、10 级组织的真实数据，秒级出结果。

为什么用 dict 不直接用类

我也想过用面向对象的方式定义 `TreeNode` 类，但有两个原因放弃了：

- **性能**——dict 的插入/查找是 $O(1)$ ，自定义类还要写 `__init__` 等模板代码
- **灵活性**——内部用 dict 存储，最终再统一转成 Pydantic 模型给前端

4.3 异常检测：路径断层

最有意思的异常是「路径断层」——比如某一行的组织路径是 [集团总部, "", 应收组]：一级有值、二级空、三级又有值。这种数据是脏的（中间断了一级），但程序需要识别出来。

检测算法用了一个简单的状态机：

```

def _detect_path_gap(path: List[str]) -> bool:
    '''检测路径是否有断层'''
    seen_non_empty = False
    seen_empty_after_non_empty = False
    for v in path:
        if v == '':
            if seen_non_empty:

```

```
        seen_empty_after_non_empty = True
    else:
        # 见到非空时，如果之前已经'非空 → 空'，
        # 那现在又出现非空，就是断层！
        if seen_empty_after_non_empty:
            return True
        seen_non_empty = True
    return False
```

这段代码的精髓是用两个布尔变量记录「状态」，遍历一次就能判断。逻辑清晰、性能 $O(n)$ 。

注意：路径末尾连续为空不算断层（很多部门没有那么深的层级），只有「非空 → 空 → 又非空」才算。

第五章 薪酬计算模块详解

5.1 业务背景：工资的构成

一个员工的月度工资计算，看起来简单，实际涉及多个环节：

应发工资 = 基本工资 + 绩效工资 + 岗位津贴 + 加班费 + 其他补贴

↓

扣除五险一金（个人部分）= 应发 × 各项费率

↓

应纳税所得额 = 应发 - 五险一金 - 5000(起征点) - 专项附加扣除

↓

个税 = 应纳税所得额 × 税率 - 速算扣除数

↓

实发工资 = 应发 - 五险一金 - 个税 - 其他扣款

5.2 五险一金费率

个人部分需要承担：

项目	个人费率	说明
养老保险	8%	固定，全国基本一致
医疗保险	2%	固定，全国基本一致
失业保险	0.5%	固定，全国基本一致
住房公积金	5%-12%	各地不同，单位可选档位

注意：五险一金的「基数」不一定等于实发工资。每个地区有最低基数和最高基数（社保上下限）。本项目简化处理，默认基数=应发工资。生产环境的薪酬系统要做更精细的处理。

5.3 个税月度税率表

中国大陆个人所得税采用累进税率，月度税率表（按月预扣预缴）：

应纳税所得额(月)	税率	速算扣除	个税示例
≤ 3000	3%	0	2000 元 → 60

3000 - 12000	10%	210	5000 元 → 290
12000 - 25000	20%	1410	20000 元 → 2590
25000 - 35000	25%	2660	
35000 - 55000	30%	4410	
55000 - 80000	35%	7160	
> 80000	45%	15160	100000 元 → 29840

速算扣除数的作用是简化计算。比如 20000 元的应纳税所得额：

- 不用速算扣除： $3000 \times 3\% + 9000 \times 10\% + 8000 \times 20\% = 90 + 900 + 1600 = 2590$
- 用速算扣除： $20000 \times 20\% - 1410 = 2590$ （结果一样，少算两步）

简化版 vs 完整版（累计预扣法）

本项目使用的是月度税率表（简化版），实际中国大陆个税预扣预缴采用的是「累计预扣法」——累计到当月的应纳税所得额查年度税率表，再减去已预缴的税额。

两者的区别：

- **简化版（本项目）** ——每月独立计算，把每月应纳税所得额套月度税率表
- **累计预扣法（生产环境）** ——年初到当月累计应纳税所得额套年度税率表，减去已预缴税额

简化版适合单月试算演示，全年累计的精确预扣需要真实的薪酬系统支持。我在文档里明确标注了这是简化版，体现「知道边界在哪」。

5.4 关键技术：Decimal 而不是 float

这是整个薪酬模块最重要的技术决策。

Python 的 float（浮点数）有精度问题：

```
>>> 0.1 + 0.2
0.30000000000000004
```

```
>>> 0.1 + 0.2 == 0.3
False
```

```
>>> 1.1 + 2.2
3.3000000000000003
```

原因：计算机用二进制存储数字，0.1 这种十进制数在二进制下是无限循环小数，只能近似存储。算钱用 float，时间一长就会出现「工资合计差几分钱」的问题。

Python 内置的 Decimal 类型专门解决这个问题：

```
from decimal import Decimal

>>> Decimal('0.1') + Decimal('0.2')
Decimal('0.3')

>>> Decimal('0.1') + Decimal('0.2') == Decimal('0.3')
True
```

所以薪酬计算引擎全程用 Decimal：

```
from decimal import Decimal, ROUND_HALF_UP

# 常量也用 Decimal
TAX_THRESHOLD = Decimal('5000')
PENSION_RATE = Decimal('0.08')

# 计算保留两位小数，四舍五入
def _round2(value: Decimal) -> Decimal:
    return value.quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)
```

做金额计算的人都该把这个原则刻在脑子里：钱用 *Decimal*，不用 *float*。这是金融、财务、电商行业的行规。

对外接口为什么转回 float

内部用 Decimal，但对外的 JSON 接口里转成 float。原因：JSON 标准没有 Decimal 类型，强行序列化会变成字符串，前端处理麻烦。

```
# 内部 Decimal 计算
result = calculate_payroll(input_data)

# 接口层转换为 float 给前端
```

```
return PayrollCalcResponse(  
    gross_salary=float(result.gross_salary),  
    net_salary=float(result.net_salary),  
)
```

这种「内部精确 + 外部友好」的设计模式很常见。

5.5 批量算薪的设计

批量算薪本质上是「循环调用单人计算」，但有几个细节：

错误容错：单行失败不影响整体

```
for idx, row in df.iterrows():  
    try:  
        result = calculate_payroll(input_data)  
        rows.append(BatchPayrollRow(success=True, result=result))  
        success_count += 1  
    except Exception as e:  
        # 一行失败，记录错误，继续算下一行  
        rows.append(BatchPayrollRow(success=False, error=str(e)))  
        failed_count += 1
```

为什么这样设计？想象一个场景：你给 500 个人算薪，第 50 个人数据有错。如果遇到错误就整个抛异常，前 49 个人的计算结果就白算了。容错设计保证：算到哪个人挂了，其他人都已经成功算出来，最后给一份「成功 + 失败」的混合报告。

结构化输出：3 个 sheet 的 Excel 报告

批量算薪的输出 Excel 包含 3 个 sheet：

- **Sheet 1: 总览**——总人数、应发总额、五险一金总额、个税总额、实发总额
- **Sheet 2: 工资条明细**——每个成功计算的员工，一行完整工资条（含 17 个字段）
- **Sheet 3: 异常清单**——所有失败的行，附带错误原因

这种「总览 + 明细 + 异常」的三层结构，是实施顾问交付给客户报告的标准格式。客户拿到这个 Excel，从领导（看总览）到经办人（看明细）到 IT（看异常）都有自己关心的页签。

第六章 工程实践

6.1 类型安全：Pydantic 的价值

用 FastAPI 不写 Pydantic 模型也能跑，但那就浪费了它最大的优势。Pydantic 让接口的「形状」被严格定义：

```
from pydantic import BaseModel, Field

class PayrollCalcRequest(BaseModel):
    '''工资计算请求'''
    base_salary: float = Field(0, ge=0, description='基本工资')
    performance: float = Field(0, ge=0, description='绩效工资')
    housing_fund_rate: float = Field(
        0.07, ge=0.05, le=0.12,
        description='公积金费率，范围 5%-12%'
    )
```

这一段代码同时做了 4 件事：

- **类型约束**——base_salary 必须是数字，不是数字会自动报 422
- **范围约束**——ge=0 表示必须 ≥ 0 ；le=0.12 表示必须 ≤ 0.12
- **默认值**——前端不传公积金费率时默认 0.07
- **自动文档**——description 会显示在 Swagger 文档里

不用 Pydantic 的话，这 4 件事都要手写代码完成。用了 Pydantic 后，5 行代码搞定全部。

6.2 单元测试：40+ 测试的安全网

这是我第一次认真为项目写测试。之前总觉得「代码能跑就行」，写测试是浪费时间。这次写完改变了想法。

项目里的测试结构：

```
tests/
├─ test_id_card.py      # 身份证算法（8 个测试）
├─ test_org_tree.py     # 组织架构解析（11 个测试）
├─ test_payroll.py      # 薪酬计算（14 个测试）
└─ test_batch_payroll.py # 批量算薪（7 个测试）
```

测试用例的设计原则

一个好的测试应该覆盖：

- **正常路径**——典型场景，看主流程对不对
- **边界场景**——刚好踩在临界点上，比如个税刚好 3000 元
- **异常输入**——空值、负数、错误格式等
- **已修复的 bug**——以前踩过的坑，写测试防止回归

比如身份证测试的设计：

```
def test_age_before_birthday():  
    '''测试生日还没到的边界场景'''  
    valid = _make_valid_id('19900307', '881')  
    # 生日前一天  
    info = id_card.parse(valid, today=date(2025, 3, 6))  
    assert info.age == 34  
    # 生日当天  
    info = id_card.parse(valid, today=date(2025, 3, 7))  
    assert info.age == 35
```

这里特别要说的设计——today 参数可注入。这样测试不会随时间漂移（如果用 date.today(), 今天测对了明年再跑就错了）。这种「可注入的依赖」是写可测试代码的关键技巧。

写测试的实际收益

写完测试后，我修改代码时的心态完全变了：

- **改之前**——心里没底，怕改坏其他地方
- **改之后跑 pytest**——8 个测试全过，安心
- **测试挂了**——明确告诉我哪个场景被改坏了，不用 print 满天飞

测试不是为了通过测试，是为了让你敢改代码。

6.3 错误处理：用户友好 vs 程序员友好

校验报告里每个错误都包含 4 个字段：

```
{
  'row': 5,                # 错在哪一行
  'field': '性别',         # 哪个字段
  'level': 'error',        # 严重程度
  'message': '性别与身份证不符', # 描述
  'suggestion': '请核对性别'   # 怎么修复
}
```

message 字段说明「出了什么问题」， suggestion 字段说明「应该怎么办」。后者尤其重要——实施顾问看到报错时不需要再思考「那我该怎么改」，提示已经在那里了。

好的错误信息和不好的错误信息对比：

✘ 不好的错误	☑ 好的错误
Validation Error	第 5 行：身份证号校验码错误
字段错误	第 3 行 工号为空，建议：请填写工号
Invalid format	第 7 行 手机号格式不正确，应为 11 位

好的错误信息包含三个要素：位置 + 现象 + 建议。

第七章 关键技术决策回顾

回头看整个项目，有几个关键决策值得单独回顾——既包括做对的，也包括差点做错被刹车的。

7.1 决策一：FastAPI vs Django

选择：FastAPI

理由：项目是纯 API 服务，不需要 Django 的模板、Admin、ORM 全家桶。FastAPI 更轻、文档自动生成、类型校验自带。

回头看：选对了。如果用 Django，光是配置就要多花 1-2 天。

7.2 决策二：金额用 Decimal 不用 float

选择：Decimal

理由：避免浮点数精度问题，这是金融行业标准。

回头看：必须的。如果用 float，单元测试都不会通过。

7.3 决策三：字段别名机制

选择：用字典定义所有字段的别名变体

理由：客户数据字段名永远不一致，硬编码「工号」会让工具完全没法用。

回头看：第一次用真实 6017 行数据跑校验时，因为别名表只覆盖了「工号」没覆盖「员工编号」，跑出 107 个「工号为空」的错误（全是误报）。修复别名表后才看到真实异常数 259。这个机制是项目的灵魂之一。

7.4 决策四：dtype=str 强制按字符串读 Excel

选择：pd.read_excel(file_path, dtype=str)

理由：避免 pandas 自动类型转换破坏数据。

回头看：必须的。如果让 pandas 自动判断类型：

- 身份证号 110101199003078812 会变成 1.10E+17（科学计数法）
- 工号 0001 会变成数字 1（前导 0 丢失）
- 日期字段会变成 Timestamp 对象，跟字符串比较时报错

全部按字符串读，需要数字时再 `parse_decimal()`，是最稳的做法。

7.5 决策五：差点做错——过度设计

差点做的事：组织架构解析模块原本要做 4 种异常检测——路径断层、空组织、同名多级、字段相似度匹配（识别「财务部」vs「财务中心」是不是同一个）。

为什么砍掉：后两个看着「高级」，但实际作为实习生项目过于复杂，会让项目失焦。AI 助手在我开始失控时多次提醒「实习生项目的常见陷阱是永远在加功能、永远没有发布的那天」。

回头看：砍得对。砍掉后核心功能更聚焦、文档更清晰、能在合理时间内交付。

做项目最难的不是「加功能」，是「决定不加功能」。这个项目教会我前者用代码实现，后者用判断力实现。

7.6 决策六：测试用例的「今天」参数可注入

选择：`id_card.parse(value, today=None)`，单元测试时可注入固定日期

理由：年龄计算依赖「今天」，如果直接调 `date.today()`，测试今天对、明天可能就错了。

回头看：写库代码时的好习惯。

第八章 HCM 业务知识补充

这一章是写给入职后还在学 HCM 基础概念的自己看的。把项目里涉及到的业务知识系统化整理一遍。

8.1 HCM 系统的核心模块

一个完整的 HCM 系统通常包含：

模块	职责
组织管理	公司、部门、岗位的层级结构
人事管理	员工档案、入职、转正、调岗、离职等全生命周期
考勤管理	打卡、请假、加班、休假
薪酬管理	薪酬体系、工资计算、个税、五险一金
绩效管理	KPI、OKR、绩效考核、绩效改进
培训管理	培训计划、课程、学习记录
招聘管理	职位发布、简历筛选、面试流程

本项目主要涉及前 4 个模块，特别是组织、人事和薪酬。

8.2 浪潮 HCM 的人员主集字段

基于真实的 01 人员信息主集.xlsx，浪潮 HCM 的人员主集包含 37 个字段，分几类：

基础信息

- 姓名、曾用名、性别、民族、国籍、籍贯、户口性质、户口所在地、出生地
- 证件类型（居民身份证/护照/港澳通行证等）、证件号
- 组织认定出生日期、连续工龄

用工信息

- 员工编号（工号）、用工类型（正式员工/劳务派遣/实习生/外聘等）
- 参加工作时间、入职日期、任职生效时间
- 岗位状态（在职/试用/挂职/借调/休假/离职）

组织信息

- 岗位_一级组织、岗位_二级组织 ... 岗位_十级组织（最多 10 级层级）
- 档案所在机构（全称）

变动信息

- 变动事务、变动原因、涉密人员标识

理解这 37 个字段的业务含义，是做好实施顾问的基础。本项目的员工档案校验模块就是基于这个数据结构设计的。

8.3 个税计算的两种方式

方式一：月度税率表（本项目使用）

每月独立计算：

$$\begin{aligned} \text{应纳税所得额(月)} &= \text{应发} - \text{五险一金} - 5000 - \text{专项附加扣除} \\ \text{个税(月)} &= \text{应纳税所得额(月)} \times \text{月度税率} - \text{月度速算扣除} \end{aligned}$$

优点：简单、易理解、单月可独立计算

缺点：跨月年终汇算时可能需要补税或退税

方式二：累计预扣法（生产环境使用）

年初到当月累计计算：

$$\begin{aligned} \text{累计应纳税所得额} &= \text{1 月到当月的累计应发} \\ &\quad - \text{累计五险一金} \\ &\quad - \text{累计起征点(月数} \times 5000) \\ &\quad - \text{累计专项附加扣除} \end{aligned}$$

$$\begin{aligned} \text{累计应纳税额} &= \text{累计应纳税所得额} \times \text{年度税率} \\ &\quad - \text{年度速算扣除} \end{aligned}$$

本月个税 = 累计应纳税额 - 已预缴税额

优点：跟年终汇算结果一致，不需要补退税

缺点：需要记录历史月份数据，单月不能独立算

本项目作为单月试算工具，使用简化的月度税率表已足够。如果未来要做成真实的薪酬系统，需要升级为累计预扣法。

8.4 五险一金的地区差异

虽然个人费率全国基本一致，但还有几个变量在各地不同：

- **社保基数上下限**——每个地区每年公布最低基数（约当地平均工资 60%）和最高基数（约 300%）
- **单位部分费率**——单位承担的部分各地差异较大（养老 16%-20%，医疗 6%-10% 不等）
- **公积金费率**——单位和个人可在 5%-12% 之间协商选择，但比例必须一致
- **生育保险**——很多地区已并入医疗保险，但有些地区仍独立缴纳
- **大病医疗 / 补充医疗**——部分地区有，部分没有

生产级的薪酬系统需要按地区配置费率。本项目简化处理，全部用全国常见值。

第九章 AI 协作的反思

这个项目大部分代码由 AI 助手（Claude）生成，但所有方向决策由我把握。做完这个项目，我对「人和 AI 协作」有一些真切的感受。

9.1 AI 适合做什么

AI 在这个项目里展现的最大价值：

- **模板代码加速**——FastAPI 路由、Pydantic 模型、pytest 用例这些有固定模式的代码，AI 写得又快又准
- **错误诊断**——遇到 UnicodeDecodeError、ModuleNotFoundError 这种报错，AI 能从错误信息直接定位原因
- **算法实现**——身份证校验、组织树构建、个税计算这些算法，AI 能直接给出正确实现
- **教学讲解**——为什么用 Decimal、为什么 dtype=str、什么是累计预扣法，AI 能讲清楚来龙去脉

9.2 AI 不适合做什么

- **决定范围**——AI 默认会「尽可能完整」，给出超出实习生项目范畴的方案。我必须主动说「砍掉」
- **业务取舍**——做不做、做到什么程度，这种基于「个人定位和目标」的判断，AI 替代不了
- **真实场景验证**——AI 写的代码看着对，但能不能用真实数据跑通，必须我自己上手测

9.3 与 AI 协作的几个原则

过程中我总结的几个原则：

原则一：把目标定清楚，再让 AI 干活。模糊的目标会让 AI 产出模糊的代码。

原则二：定边界。说「不要做 XX」和说「要做 XX」同样重要。

原则三：每个代码块都自己跑一遍。不要假设「AI 说能跑就能跑」。

原则四：理解每一行代码的意图。否则项目就是「AI 借我的手输出的」，不是「我的」。

原则五：拒绝过度设计。AI 容易「贪多」，要主动刹车。

9.4 一个具体的案例

项目中段，我说「再加一个功能吧」，AI 没有立刻答应，反而停下来反问：

回想我们的对话节奏：v1.0、v1.5、v2.0 每次你都说「再加一个」。我担心：不管做到什么程度，你都会觉得「还差点」，然后无限循环下去。实习生项目的常见陷阱：项目永远在「再加一个就好」，永远没有发布的那天。

这次反问让我停下来认真思考「我做这个项目的真实目的」。最后达成共识：不是为了找工作，而是为了在浪潮立足、用代码理解 HCM 业务。

这一刻让我意识到：好的 AI 助手不是「无条件听话的工具」，而是「会反对你的协作者」。

这种反对让项目少走了至少一周的弯路。

第十章 后续规划与反思

10.1 短期计划 (v3.0)

- **通用能力模块**——字段映射器（可视化配置源字段到 HCM 字段的映射）
- **校验规则库**——把硬编码的校验规则改为可配置的规则库
- **操作日志**——每次校验都记录日志，方便追溯

10.2 中期计划 (v3.5)

- **累计预扣法**——把月度个税升级为真实的累计预扣
- **数据迁移 workflow**——一键串联校验、解析、薪酬试算，输出完整的「客户数据迁移准备度报告」
- **部署到云端**——让项目有一个真正的线上 demo URL

10.3 长期计划 (v4.0+)

- **用户系统 + 项目管理**——支持多客户、多项目
- **反哺真实场景**——等我接触到一线项目，把真实痛点反哺到工具里

10.4 给未来自己的话

写这份文档的初衷之一，是给未来某天回看的自己留一份提醒。所以最后这一段，我想直接对那个未来的自己说：

如果你正在为某段代码烦恼，回头看这份文档。每一行代码当时都有理由——也许那个理由现在看起来已经不对了，那很好，说明你成长了。

如果你已经离开了 HCM 行业，去做了别的，记得这个项目教会你的不是技术，而是「在没有任务时给自己造一个任务」的能力。这种能力在任何行业都管用。

如果你还在做实施顾问，希望你已经成为既懂业务又懂技术的那种人。希望你接触到的每

一个客户、每一份数据、每一次实施，都让你比上一次更游刃有余。

无论如何，2026 年 5 月的那两周，你做了一件认真的事。这件事不必伟大，但它是你的。

—— 写于 2026 年 5 月

宋庚 (Geng Song)